

DEEP REINFORCEMENT LEARNING

365.382 (SS26)

Policy-Based Methods



Sohvi Luukkonen

LIT AI LAB

Institute for Machine Learning

Resources

- [Reinforcement Learning: An Introduction](#) [Chapter 13]
- [DeepMind x UCL | RL Lecture Series 2021](#) [Lecture 9]

Introduction and Motivation

RL Taxonomy

■ Value-based

- ☐ Learned value function
- ☐ Implicit policy (ϵ -greedy)

■ Policy-based

- ☐ Learned policy
- ☐ No value function

■ Actor-Critic

- ☐ Learned policy (actor)
- ☐ Learned value function (critic)

Value-Based RL

- Learn a value function first and derive a deterministic policy indirectly via

$$\pi(s) = \operatorname{argmax}_a q(s, a)$$

- Tabular value-based methods:

- ☐ $q(a, s)$ for each state-action pair
- ☐ MC Control, SARSA, Q-learning
- ☐ Work well for small discrete state and action spaces

- Value function approximation methods:

- ☐ $q_{\mathbf{w}}(s, a)$ with parameters $\mathbf{w}$
- ☐ Linear function approximation, DQN
- ☐ Handle large or continuous state spaces

Problem 1: Continuous Action Spaces

In many control problems, actions are not discrete choices.

- Robot torque: $a \in \mathbb{R}^d$
- Steering, throttle, braking in autonomous driving
- Motor commands for locomotion or manipulation

For value-based methods, action selection requires solving

$$a^*(s) = \operatorname{argmax}_a q(s, a)$$

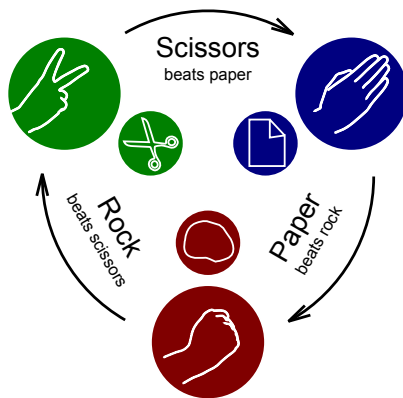
which is easy for a few discrete actions, but difficult in continuous spaces.

Problem 2: Deterministic Greedy Policies Can Be Limiting

Greedy improvement in value-based RL naturally leads to deterministic behavior.

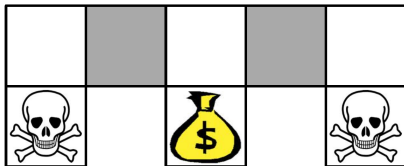
- But in some tasks, a **stochastic policy** is useful or necessary
- Exploration can be represented explicitly by the policy itself
- In partially observable settings, stochastic policies can be better than deterministic ones
- Some environments may have multiple equally good actions, where randomization is natural

Example: Rock-Paper-Scissors



- A deterministic policy (e.g., always play rock) is easily exploited
- A uniform random policy is optimal (i.e., Nash equilibrium)

Example: Aliased Gridworld



- Each state is described by feature vector $\phi(s)$ indicating the presence of walls in each direction

$$\phi(s) = [\text{wall up}, \text{wall right}, \text{wall down}, \text{wall left}]$$

- The agent cannot differentiate gray states
- Non-Markovian environment: optimal action depends on history, not just current state

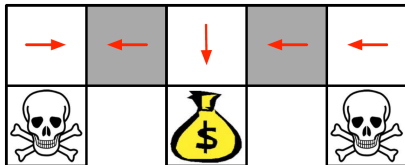
Interlude: Partially Observable MDPs (POMDPs)

- So far, we assumed **fully observable** MDPs:
 - the agent observes the true state S_t
- In many problems, the agent only observes an **observation** O_t
 - $O_t \sim p(o | S_t)$
 - the true state S_t is hidden
- This setting is called a **Partially Observable MDP (POMDP)**
- The process is no longer Markov w.r.t. observations:
 - $p(O_{t+1} | O_t, A_t)$ depends on hidden state S_t
 - optimal decisions may depend on **history**, not just O_t

POMDPs: Consequences

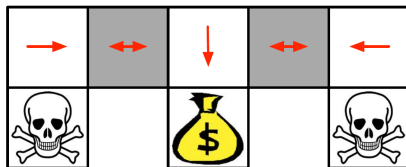
- Aliased Gridworld is a simple example of partial observability:
 - different states share the same observation $\phi(s)$
- Consequences:
 - optimal policy may need **memory** (history-dependent)
 - or **randomization** (stochastic policy)
- In this course:
 - we will mostly ignore memory
 - but allow **stochastic policies**
- $\Rightarrow$ policy-based methods can naturally represent such policies

Example: Aliased Gridworld – Deterministic Policy



- Under aliasing, an optimal **deterministic** policy will either
 - ☐ move left in both gray states (shown in figure), or
 - ☐ move right in both gray states
- Either way, the agent can get stuck and never reach the goal

Example: Aliased Gridworld – Stochastic Policy



- An optimal **stochastic** policy will move randomly left or right in the gray states

$$\pi_{\theta}(\text{left} \mid \text{wall up and down}) = \pi_{\theta}(\text{right} \mid \text{wall up and down}) = 0.5$$

- Reaches the goal in few states with high probability, and never gets stuck
- By directly learning the policy parameters θ , we can learn an optimal stochastic policy

Advantages of Policy-Based Methods

- True objective
- Natural for high-dimensional and continuous action spaces
- Can learn stochastic policies
- Sometimes policies are simple while value functions are complex
 - e.g., complicated dynamics but simple optimal behavior (e.g., "always go left")
- Choice of policy class can encode prior knowledge and bias learning towards good solutions

Disadvantages of Policy-Based Methods

- Often higher variance than value-based methods
- Can be less sample efficient
- Can get stuck in local optima
- Obtained knowledge can be specific, does not always generalize well

Policy Optimization

Direct Policy Learning

- Goal: given a policy π_θ with parameters θ , find the optimal parameters θ^*
- How do we measure the quality of a policy π_θ ?
- In episodic environments we can use the average total return per episode
- In continuing environments we can use the average reward per time step

Policy Objective Functions – Episodic

■ Episodic-return objective:

$$\begin{aligned} J(\theta) &= \mathbb{E}_{S_0 \sim d_0, \pi_\theta} \left[\sum_{t=0}^T \gamma^t R_t \right] \\ &= \mathbb{E}_{S_0 \sim d_0, \pi_\theta} [G_0] \\ &= \mathbb{E}_{S_0 \sim d_0} [\mathbb{E}_{\pi_\theta} [G_t \mid S_t = S_0]] \\ &= \mathbb{E}_{S_0 \sim d_0} [v_{\pi_\theta}(S_0)] \\ &= \mathbb{E}_{S_0 \sim d_0} \left[\sum_a \pi_\theta(a \mid S_0) q_{\pi_\theta}(S_0, a) \right] \end{aligned}$$

where d_0 is the distribution over initial states.

■ Equals to the expected value of the start state.

Policy Objective Functions – Average Reward

■ Average-reward objective:

$$\begin{aligned} J(\theta) &= \mathbb{E}_{\pi_{\theta}} [R_{t+1}] \\ &= \mathbb{E}_{S_t \sim d_{\pi_{\theta}}} \left[\mathbb{E}_{\pi_{\theta}} [R_{t+1} \mid S_t] \right] \\ &= \sum_s d_{\pi_{\theta}}(s) \sum_a \pi_{\theta}(a \mid s) \sum_r p(r \mid s, a) r \end{aligned}$$

where $d_{\pi_{\theta}}(s) = p(S_t = s \mid \pi_{\theta})$ is the probability of visiting state s under policy π_{θ} (ratio of time spent in s).

- Equals to the expected value of the reward in the states visited by the policy.

Policy Optimization

- Policy-based RL is an **optimization** problem: find $\theta^* = \operatorname{argmax}_{\theta} J(\theta)$
- Many algorithms for optimizing policies
- Gradient-free methods:
 - ☐ Hill climbing
 - ☐ Simplex / “Amoeba” / Nelder-Mead
 - ☐ Genetic algorithms
- Gradient-based methods:
 - ☐ **Gradient ascent**
 - ☐ Conjugate gradient
 - ☐ Quasi-Newton methods

Policy Gradients

Policy Gradient

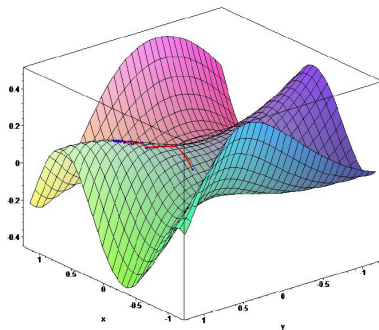
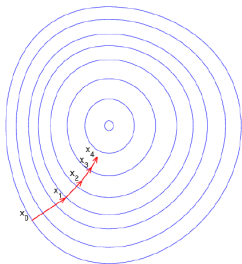
- Search for **local maximum** by **ascending** the gradients of the objective $J(\theta)$, w.r.t the policy parameters θ :

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$$

- where α is the learning rate, and
- $\nabla_{\theta} J(\theta)$ is the **policy gradient**

$$\nabla_{\theta} J(\theta) = \left(\frac{\partial J(\theta)}{\partial \theta_1}, \frac{\partial J(\theta)}{\partial \theta_2}, \dots, \frac{\partial J(\theta)}{\partial \theta_n} \right)^{\top}$$

- Stochastic policies help ensure that $J(\theta)$ is (mostly) smooth



Gradient on Parametrized Policies

- How do we compute the policy gradient $\nabla_{\theta} J(\theta)$?
- Finite difference methods: estimate the gradient by perturbing the parameters and measuring the change in $J(\theta)$
- Analytic methods: derive a formula for the gradient using calculus and the structure of the MDP

Note: today we only focus, on **model-free** policy gradients, which do not require knowledge of the environment dynamics.

Policy Gradient by Finite Difference

- For each dimension i , we can estimate the gradient by perturbing the parameter in the i -th dimension:

$$\frac{\partial J(\theta)}{\partial \theta_i} \approx \frac{J(\theta + \epsilon e_i) - J(\theta - \epsilon e_i)}{\epsilon}$$

where e_i is the unit vector in the i -th direction and ϵ is a small perturbation.

- Simple to implement, but requires $O(n)$ evaluations of $J(\theta)$ for n parameters.
- Simple, noisy, and inefficient for high-dimensional policies.
- Works for arbitrary policies, even if policy is not differentiable.

Score Function

- From now on, assume policy π_θ is differentiable almost everywhere w.r.t θ .
- and we know the gradient $\nabla_\theta \pi_\theta(s, a)$ of the policy.
- **Likelihood ratios** exploit the following identity:

$$\begin{aligned}\nabla_\theta \pi_\theta(s, a) &= \pi_\theta(s, a) \frac{\nabla_\theta \pi_\theta(s, a)}{\pi_\theta(s, a)} \\ &= \pi_\theta(s, a) \underbrace{\nabla_\theta \log \pi_\theta(s, a)}_{\text{score function}}\end{aligned}$$

Example: Softmax Policy

- The probability of selecting action a in state s is proportional to the exponentiated preference $h(s, a)$ (a stochastic policy):

$$\pi_{\theta}(a \mid s) = \frac{e^{h_{\theta}(s, a)}}{\sum_b e^{h_{\theta}(s, b)}}$$

- The gradient of the log-probability (score function) is:

$$\nabla_{\theta} \log \pi_{\theta}(A_t \mid S_t) = \underbrace{\nabla_{\theta} h_{\theta}(S_t, A_t)}_{\text{preference gradient}} - \underbrace{\sum_a \pi_{\theta}(a \mid S_t) \nabla_{\theta} h_{\theta}(S_t, a)}_{\text{expected preference gradient}}$$

- Action preferences $h_{\theta}(s, a)$ can be parameterized arbitrarily.
- If $h_{\theta}(s, a) = \theta^{\top} \phi(s, a)$, the score function simplifies to:

$$\nabla_{\theta} \log \pi_{\theta}(A_t \mid S_t) = \phi(S_t, A_t) - \mathbb{E}_{\pi_{\theta}}[\phi(S_t, \cdot)]$$

One-Step MDPs (Contextual Bandits)

- Consider a one-step MDP (contextual bandit) such that $J(\theta) = \mathbb{E}_{\pi_\theta}[R_{t+1}]$.
 - Starts in state $S_t \sim d(s)$
 - Terminating after one step, with reward $R_{t+1} \sim p(r \mid S_t, A_t)$
- Can not sample R_{t+1} and then take the gradient, because R_{t+1} is just a number not a function of θ .
- Instead, we can use the score function to compute the policy gradient:

$$\nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}_{\pi_\theta}[R_{t+1}] = \mathbb{E}_{\pi_\theta}[R_{t+1} \nabla_\theta \log \pi_\theta(A_t \mid S_t)]$$

- Now we have an expected gradient that we can sample, and **stochastic policy-gradient** update is:

$$\theta \leftarrow \theta + \alpha R_{t+1} \nabla_\theta \log \pi_\theta(A_t \mid S_t)$$

Score function / Log-likelihood trick / REINFORCE

$$\begin{aligned}\nabla_{\theta} \mathbb{E}_{\pi_{\theta}}[R] &= \nabla_{\theta} \sum_s d(s) \sum_a \pi_{\theta}(a | s) r_{s,a} \\&= \sum_s d(s) \sum_a r_{s,a} \nabla_{\theta} \pi_{\theta}(a | s) \\&= \sum_s d(s) \sum_a r_{s,a} \pi_{\theta}(a | s) \frac{\nabla_{\theta} \pi_{\theta}(a | s)}{\pi_{\theta}(a | s)} \\&= \sum_s d(s) \sum_a r_{s,a} \pi_{\theta}(a | s) \nabla_{\theta} \log \pi_{\theta}(a | s) \\&= \mathbb{E}_{d, \pi_{\theta}} [R \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)]\end{aligned}$$

Policy Gradient Theorem

Policy Gradient Theorem

- The policy gradient approach extends from bandits to multi-step MDPs
- Objective:
 - maximize expected long-term return under a parameterized policy π_θ
- Main challenge:
 - actions influence not only immediate rewards, but also future states and future rewards
- The policy gradient theorem gives a surprisingly simple result:
 - the gradient can be written without differentiating through the environment dynamics
- Two policy gradient formulations for **average return per episode** and **average reward per step**

Policy Gradient Theorem (Episodic)

Theorem

For any differentiable policy $\pi_\theta(a \mid s)$, let d_0 be the starting-state distribution. Then the policy gradient of $J(\theta) = \mathbb{E}_{d_0, \pi_\theta}[G_0]$ is

$$\nabla_\theta J(\theta) = \mathbb{E}_{d_0, \pi_\theta} \left[\sum_{t=0}^T \gamma^t q_{\pi_\theta}(S_t, A_t) \nabla_\theta \log \pi_\theta(A_t \mid S_t) \right]$$

where

$$\begin{aligned} q_{\pi_\theta}(s, a) &= \mathbb{E}_{\pi_\theta}[G_t \mid S_t = s, A_t = a] \\ &= \mathbb{E}_{\pi_\theta}[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a] \end{aligned}$$

- Policy gradients do **not** require differentiating through the MDP dynamics

Episodic Policy Gradients: Proof (I)

- Consider trajectory $\tau = (S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_T)$ with return $G(\tau)$
- The policy gradient can be written as

$$\begin{aligned}\nabla_{\theta} J(\theta) &= \nabla_{\theta} \mathbb{E}_{\pi_{\theta}}[G(\tau)] \\ &= \nabla_{\theta} \sum_{\tau} p_{\pi_{\theta}}(\tau) G(\tau) \\ &= \sum_{\tau} G(\tau) \nabla_{\theta} p_{\pi_{\theta}}(\tau) \\ &= \sum_{\tau} p_{\pi_{\theta}}(\tau) G(\tau) \nabla_{\theta} \log(p_{\pi_{\theta}}(\tau)) \\ &= \mathbb{E}_{\pi_{\theta}} [G(\tau) \nabla_{\theta} \log p_{\pi_{\theta}}(\tau)]\end{aligned}$$

Episodic Policy Gradients: Proof (II)

■ Let's expand the trajectory probability:

$$\begin{aligned}\nabla_{\theta} \log p_{\pi_{\theta}}(\tau) &= \nabla_{\theta} \log \left(p(S_0) \prod_{t=0}^T \underbrace{p(S_{t+1} \mid S_t, A_t)}_{\text{model}} \underbrace{\pi_{\theta}(A_t \mid S_t)}_{\text{policy}} \right) \\ &= \nabla_{\theta} \left(\log p(S_0) + \sum_{t=0}^T \log p(S_{t+1} \mid S_t, A_t) + \sum_{t=0}^T \log \pi_{\theta}(A_t \mid S_t) \right) \\ &= \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(A_t \mid S_t)\end{aligned}$$

□ The initial state distribution $p(S_0)$ and the transition probabilities $p(S_{t+1} \mid S_t, A_t)$ do not depend on θ , so their gradients are zero.

Episodic Policy Gradients: Proof (III)

$$\begin{aligned}
 \nabla_{\theta} J(\theta) &= \mathbb{E}_{\pi_{\theta}} \left[G(\tau) \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right] = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^T G(\tau) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right] \\
 &= \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^T \left(\sum_{k=0}^T \gamma^k R_{k+1} \right) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right] \quad \begin{array}{l} R_{k+1}, k < t \text{ do not depend on } A_t \\ \Rightarrow \mathbb{E}[R_{k+1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)] = 0 \end{array} \\
 &= \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^T \left(\sum_{k=t}^T \gamma^k R_{k+1} \right) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right] \\
 &= \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^T \left(\gamma^t \sum_{k=t}^T \gamma^{k-t} R_{k+1} \right) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right] \\
 &= \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^T (\gamma^t G_t) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right] = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^T \gamma^t q_{\pi_{\theta}}(S_t, A_t) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right]
 \end{aligned}$$

How Can We Use the Theorem?

- The theorem gives the gradient in terms of $q_{\pi_\theta}(S_t, A_t)$:

$$\nabla_\theta J(\theta) = \mathbb{E}_{d_0, \pi_\theta} \left[\sum_{t=0}^T \gamma^t q_{\pi_\theta}(S_t, A_t) \nabla_\theta \log \pi_\theta(A_t | S_t) \right]$$

- Problem: $q_{\pi_\theta}(s, a)$ is usually unknown
- In episodic problems, after taking action A_t in state S_t , we can observe the return G_t
- Since

$$q_{\pi_\theta}(s, a) = \mathbb{E}_{\pi_\theta}[G_t | S_t = s, A_t = a]$$

the return G_t is an unbiased sample of $q_{\pi_\theta}(S_t, A_t)$

- Replace:
 - expectation $\rightarrow$ sample trajectory
 - $q_{\pi_\theta}(S_t, A_t) \rightarrow G_t$

Monte Carlo Policy Gradient (REINFORCE)

- Using return G_t as an unbiased estimate of $q_{\pi_\theta}(s, a)$, and stochastic gradient ascent, we get the REINFORCE update:

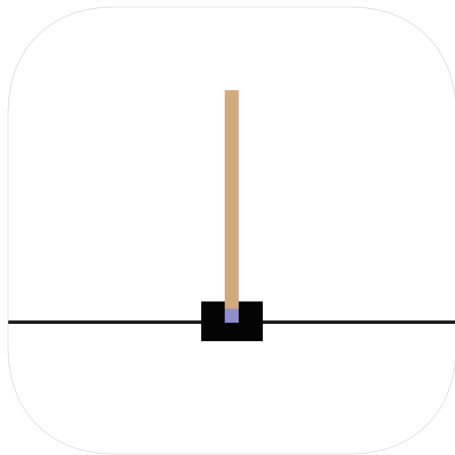
$$\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla_\theta \log \pi_\theta(A_t | S_t)$$

Algorithm: REINFORCE

- 1: Initialize policy parameters θ
- 2: **for** each episode $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_T \sim \pi_\theta$ **do**
- 3: **for** each step t in the episode **do**
- 4: Compute return $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-t-1} R_T$
- 5: Update policy parameters: $\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla_\theta \log \pi_\theta(A_t | S_t)$
- 6: **end for**
- 7: **end for**
- 8: **return** policy parameters θ

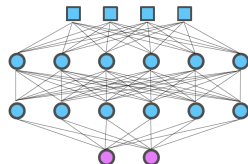
Example: CartPole

- Balance a pole on a cart by applying forces left or right
- State: $S_t =$
(position, speed, angle, angle-speed)
- Action: $A_t \in \{\text{left}, \text{right}\}$
- Reward: $R_{t+1} = 1$ for every step the pole is balanced, episode ends when pole falls or after 200 steps



Example: CartPole

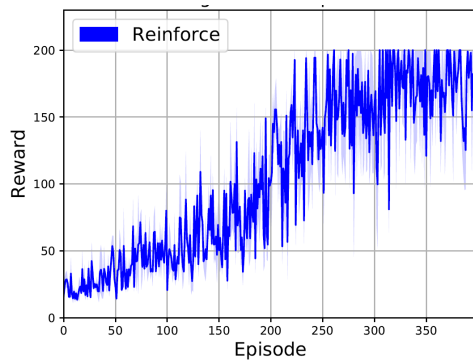
Position, Speed, Angle, Angle-Speed



four input neurons

two layer neural network
with 32 hidden units and
tanh-nonlinearity

one output
neuron per action
+
softmax non-linearity



Reduce Variance

- Note, that in general,

$$\begin{aligned}\mathbb{E}[b \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)] &= \mathbb{E}\left[\sum_a \pi_{\theta}(a|S_t) b \nabla_{\theta} \log \pi_{\theta}(a|S_t)\right] \\ &= \mathbb{E}\left[b \nabla_{\theta} \sum_a \pi_{\theta}(a|S_t)\right] \\ &= \mathbb{E}[b \nabla_{\theta} 1] = 0\end{aligned}$$

- True, if b doesn't depend on the action (but it can depend on the state)
- We can subtract a baseline $b(S_t)$ from the return without changing the expected gradient, but potentially reducing the variance of the updates.

REINFORCE with Baseline

- MC policy gradient suffers from high variance
- Subtracting a state-dependent **baseline** $b(S_t)$ leaves the expected gradient unchanged but reduces variance:

$$\theta \leftarrow \theta + \alpha (G_t - b(S_t)) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$$

- $b(s) = 0$ recovers plain REINFORCE
- Baseline should vary with the state, but not with the action
- Common choice: $b(s) = v_{\mathbf{w}}(s)$, a value function learned separately

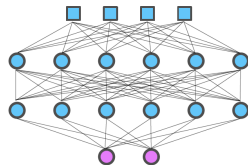
REINFORCE with Baseline

Algorithm: REINFORCE with Baseline

- 1: Initialize policy parameters θ and value function parameters $\mathbf{w}$
- 2: **for** each episode $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_T \sim \pi_\theta$ **do**
- 3: **for** each step t in the episode **do**
- 4: $G_t = \sum_{k=t}^{T-1} \gamma^{k-t} R_{k+1}$
- 5: $\theta \leftarrow \theta + \alpha_\theta \gamma^t (G_t - v_{\mathbf{w}}(S_t)) \nabla_\theta \log \pi_\theta(A_t | S_t)$
- 6: $\mathbf{w} \leftarrow \mathbf{w} + \alpha_{\mathbf{w}} (G_t - v_{\mathbf{w}}(S_t)) \nabla_{\mathbf{w}} v_{\mathbf{w}}(S_t)$
- 7: **end for**
- 8: **end for**
- 9: **return** policy parameters θ and value function parameters $\mathbf{w}$

Example: CartPole

Position, Speed, Angle, Angle-Speed



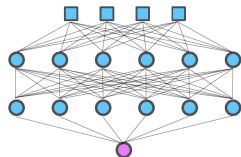
Action-selection probabilities

four input neurons

two layer neural network
with 32 hidden units and
tanh-nonlinearity

one output
neuron per action
+
softmax non-linearity

Position, Speed, Angle, Angle-Speed

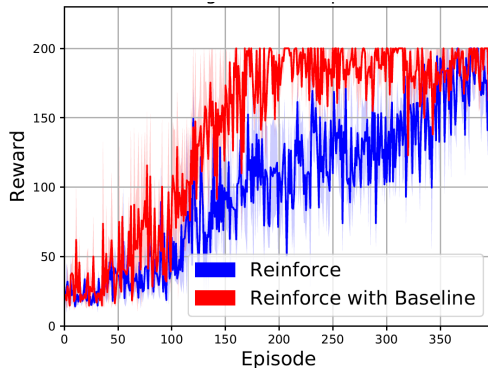


state-value estimate

four input neurons

two layer neural network
with 32 hidden units and
tanh-nonlinearity

one linear output neuron



Actor-Critic Methods

From REINFORCE to Actor-Critic

- REINFORCE with Baseline uses the value function is only used as a **baseline**
 - ☐ it reduces variance, but does **not define the learning signal**
 - ☐ no bootstrapping (uses full return G_t)
- Actor-Critic methods use the value function to **bootstrap**
 - ☐ replace the Monte Carlo return G_t with a bootstrapped estimate
 - ☐ e.g., $G_t \approx R_{t+1} + \gamma v_{\mathbf{w}}(S_{t+1})$
 - ☐ the critic now **provides the learning signal**
- This leads to:
 - ☐ lower variance and better sample efficiency
 - ☐ but introduces bias due to bootstrapping

One-step Actor-Critic

- Replace the return G_t with the one-step bootstrapped estimate $G_{t:t+1}$ in policy gradient update:

$$\begin{aligned}\theta_{t+1} &= \theta_t + \alpha_\theta \gamma^t (G_{t:t+1} - v_{\mathbf{w}}(S_t)) \nabla_\theta \log \pi_\theta(A_t | S_t) \\ &= \theta_t + \alpha_\theta \gamma^t (R_{t+1} + \gamma v_{\mathbf{w}}(S_{t+1}) - v_{\mathbf{w}}(S_t)) \nabla_\theta \log \pi_\theta(A_t | S_t) \\ &= \theta_t + \alpha_\theta \gamma^t \delta_t \nabla_\theta \log \pi_\theta(A_t | S_t)\end{aligned}$$

- Interpretation:

- ☐ δ_t measures how much better the outcome was than expected
- ☐ $\Rightarrow$ it acts as an **advantage estimate**

One-step Actor-Critic

Algorithm: One-Step Actor-Critic

- 1: Initialize policy parameters θ and value function parameters $\mathbf{w}$
- 2: **for** each episode **do**
- 3: Initialize $S_0 \sim d_0$
- 4: **for** each step t in the episode **do**
- 5: $\delta = R_{t+1} + \gamma v_{\mathbf{w}}(S_{t+1}) - v_{\mathbf{w}}(S_t)$ ▷ one-step TD error or **advantage**
- 6: $\mathbf{w} \leftarrow \mathbf{w} + \alpha_{\mathbf{w}} \delta \nabla_{\mathbf{w}} v_{\mathbf{w}}(S_t)$ ▷ semi-gradient TD(0) update
- 7: $\theta \leftarrow \theta + \alpha_{\theta} \gamma^t \delta \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$ ▷ policy gradient update
- 8: **end for**
- 9: **end for**

n -step Actor-Critic

- One-step TD uses a very short horizon
- Use an n -step bootstrapped return

$$G_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l R_{t+l+1} + \gamma^n v_{\mathbf{w}}(S_{t+n})$$

- Build advantage estimate

$$\hat{\delta}_t^{(n)} = G_t^{(n)} - v_{\mathbf{w}}(S_t)$$

- Update actor with

$$\theta \leftarrow \theta + \alpha_{\theta} \gamma^t \hat{\delta}_t^{(n)} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$$

From n-Step to GAE

- We now have many valid advantage estimates:

$$\delta_t^{(1)}, \delta_t^{(2)}, \delta_t^{(3)}, \dots$$

- Each choice of n gives a different bias–variance tradeoff
- Question:
 - ☐ can we combine them instead of choosing one?
- Idea:
 - ☐ take a weighted average of all n -step estimates

Generalized Advantage Estimation (GAE)

- Define TD errors:

$$\delta_t = R_{t+1} + \gamma v_{\mathbf{w}}(S_{t+1}) - v_{\mathbf{w}}(S_t)$$

- GAE estimator:

$$\hat{A}_t^{\text{GAE}(\lambda)} = \delta_t + \gamma\lambda\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \dots$$

- Equivalent form:

$$\hat{A}_t^{\text{GAE}(\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$$

- Interpretation:

- ☐ combines multiple n-step advantage estimates
- ☐ exponentially downweights longer horizons

Understanding GAE

- λ controls the bias–variance tradeoff
- $\lambda = 0$:
 - only uses δ_t (1-step TD)
- $\lambda = 1$:
 - corresponds to Monte Carlo advantage
- $0 < \lambda < 1$:
 - interpolates between TD and Monte Carlo

Summary

- Policy-based methods directly learn a policy, without learning a value function
- Policy gradients optimize the policy parameters by ascending the gradient of the expected return
- The policy gradient theorem provides a formula for computing the policy gradient using only samples from the environment
- Actor-critic methods use a value function to bootstrap and reduce variance, leading to more efficient learning